

CHAPTER 01

心智模型

把 AI 当工具，用一辈子也用不顺。
把 AI 当同事，三天就开窍。

今天讲什么

5 章 · 34 页 · 30–40 分钟 · 有 demo · 可截图带走

1 · 心智模型

心智 · 三层架构 · 名词速查 · Claw 家族 · Harness/Hermes · MCP 协议 · 合规边界

2 · 三同事详解

CC / WorkBuddy / CodeBuddy 速览 + 4 个真实 demo · 横向对比 · 决策矩阵

3 · 模型与 Context

Opus / Sonnet / Haiku 同事类比 · Context Window · 价格直觉 · 非 Anthropic 模型

4 · 配置即记忆

AGENTS.md why+how · Skill / Hook 最小模板

5 · 起步路径

5 条误解辟谣 · 场景速查 · 成长路径 Day 1 → Quarter 1

预期：听完之后，你能说清楚自己手头的活该用哪个 agent，知道至少 1 处可以马上节省 30 分钟的具体落地点。

讲师备注 · 开场 1 分钟。强调 "今天不会让你装东西，先听完再决定要不要上手"。

AI 不是工具，是同事。

心智一旦切对，整套用法都顺了

❌ 把 AI 当工具

- 每次都从零交代背景
- 不给上下文，问完就走
- 得到一句模糊回答，骂"AI 不行"
- 不教它，不喂资料，不写偏好
- 结果：5 分钟省下 30 秒，再花 4 分钟改

✅ 把 AI 当同事

- 给身份："我是法律行业销售"
- 给偏好："HTML 我要 Apple 风"
- 给项目地图："这个客户的历史在 X 文件"
- 明确反模式："不要给我 5 个选项让我挑"
- 结果：1 小时投入 / 半年复利

核心切换：从 "AI 帮我做 X" → 变成 "AI 知道我是谁、在哪、要去哪、之后再帮我做 X"。
这就是为什么"配置"和"记忆"是这堂课最重要的两个词。

讲师备注 · 这页是全场心智锚点。可以问听众"你最近一次用 AI 失望是什么时候？" 大概率答案都是把 AI 当工具的失败经历。



CHAPTER 02

三个同事

CC 是动手实习生，CodeBuddy 是身边小助理，WorkBuddy 是长期顾问。

三个 AI 同事 · 角色 + 市面归类

先认人 · 再看名片 · 再判断它和你常用的工具是不是一类

CC（Claude Code · Anthropic 官方 CLI · 在终端运行 · 能读写文件、跑 shell、git）

动手实习生 · 终端 CLI

载体

命令行 CLI

最强场景

批量改文件 / git

叫它

"我电脑上要动手做"

市面同类 · CLI agent

Aider · Cursor CLI · OpenAI Codex CLI · Gemini CLI · OpenCode · Goose

WorkBuddy

长期顾问 · 独立桌面 agent

载体

独立 agent 实例

最强场景

跨日策略 / 复盘

叫它

"参考过去半年的事"

市面同类 · 桌面 agent

ChatGPT Desktop · Claude Desktop · Perplexity · Notion AI · Granola（会议）

CodeBuddy

身边小助理 · IDE 插件

载体

IDE 插件

最强场景

编辑器内补全 / 解释

叫它

"在 IDE 里随手问一下"

市面同类 · IDE 插件

GitHub Copilot · Cursor · Windsurf · Cline · 通义灵码 · Comate

三同事的「记忆」机制对比

维度	CC · CLAUDE.md	CodeBuddy · User Rules + 项目文件	WorkBuddy · 自动持续沉淀
怎么写	手动写 markdown，会话注入	UI 设置里填规则 + 读项目内 .md	对话中自动写 daily memory（agent 自管）
维护成本	中 自己定期清理	低 设置一次半年不动	低 自动管理
记忆深度	会话级 + 项目级	项目级 + 全局偏好	跨日跨项目（最深）

能不能共用一份记忆？—— 部分可以。可共用：项目级 AGENTS.md（CC + CodeBuddy + Cursor 都吃）。不要共用：WorkBuddy 的 daily memory 和 SOUL 人格——它是对话沉淀，强行喂给 CC 反而污染上下文。建议：项目规则 → 一份 AGENTS.md 跨家共用；个人偏好 → CC 写 CLAUDE.md / CodeBuddy 写 User Rules（手动同步）；业务记忆 → WorkBuddy 独占。

讲师备注 · 这页停 2 分钟。市面同类那行让大家"对号入座"——他们用过 Cursor / Copilot 就秒懂 CodeBuddy 是什么。记忆机制差异是这堂课的暗线，后面 P12 会再展开 WB 三件套。

三层架构 + 常见名词速查

所有 AI agent 都长一个样 · 名词听到不再蒙

APPLICATION · 应用层

CodeBuddy · Claude Code · WorkBuddy · 微信机器人 · ...

你直接打字的对象

ROUTING · 路由层

OpenClaw 网关 · 鉴权 · 模型分发 · 设备配对 · 审批流

公司 IT 配好的“水电”

MODEL · 模型层

Anthropic Claude (Opus / Sonnet / Haiku) · 智谱 GLM · DeepSeek · OpenAI GPT · 混元

名词速查 · 听到这些词不再懵

Agent

能“自己决定下一步做什么”的 AI 实例。三同事都是 agent。

Tool / Function

AI 能调用的“动作”。读文件、发请求、跑命令都算 tool。

Token

AI 处理文本的最小单位。1 个汉字≈1.5–2 token。计费就按它算。

Prompt

你给 AI 的输入。包括 system prompt（角色设定）+ user prompt（你的话）。

Context Window

“AI 一次能装在脑子里的信息量”。200K / 1M 是上限。

RAG

检索增强 · 让 AI“先去查资料再回答”。知识库类应用都用它。

Embedding

把文本转成向量数字 · 用于“相似搜索”。RAG 的底层。

Function Calling

AI 主动决定调哪个 tool 的能力。Agent 的核心机制。

MCP

Model Context Protocol · “AI 怎么连外部”的开放协议。后面专讲。

System Prompt

藏在最前面的“角色定义”。SOUL.md / CLAUDE.md 都注入这里。

Hallucination

幻觉 · AI 编了个看起来对的答案。给上下文是最好的解药。

Fine-tuning

微调 · 在原模型上喂数据训出“专属版”。业务用通常不需要。

Inference

推理 · 模型从输入生成输出的过程。你每次“问 AI”就是一次 inference。

Streaming

流式输出 · 一边生成一边显示。所以你看 AI“打字”。

Skill

工作流模板 · 把“反复做的事”封装成 SKILL.md。一句话调用。

Hook

触发器 · “写完文件后自动跑测试”这种自动化。

讲师备注 · 这页是“扫盲页”。听众看到这些词以后就不会蒙。强调 token / context / RAG / MCP / Skill / Hook 这 6 个最常听到的，其他过一下即可。

"Claw" 这个词 · 一次说清

市面 + 腾讯内部叫 Claw 的有 4 类 · 不在同一层 · 别混淆

名字	所在层	它是什么（专业版）	大白话理解
OpenClaw	路由层	多模型路由网关 · 本机表现为 <code>~/openclaw/</code> 服务进程 · 统一鉴权 + 模型分发 + 审批流	"水电管线" · 你不用直接和它说话 · 但所有 agent 都靠它接到模型
Claw Code	应用层 CLI	Claude Code 的开源衍生 / 二次封装 · npm 安装 · 可自部署 · 兼容 CC 的 hook/skill 机制	"开源版 CC" · 给开发者用的可改可路由的 CLI · 业务同事一般不会直接接触
WorkBuddy Claw	应用层 Agent	WorkBuddy 桌面实例的品牌昵称 · 独立 agent 进程 · 自带 SOUL/IDENTITY/USER 人格 + 自动 daily memory	"长期顾问的英文名字" · 听到 "WorkBuddy Claw" = 听到 "WorkBuddy"
CodeBuddy 内的 Claw 标签	应用层插件特性	CodeBuddy IDE 中标识"接入 OpenClaw 网关 / 走 Claw 模型组合"的能力开关	"CodeBuddy 的一个开关" · 开了之后它走 OpenClaw 路由，不开就走默认

关键判断：听到 "Claw" 怎么反应

- 同事说"用 Claw 帮我写" → 八成指 WorkBuddy Claw 或 Claw Code，问一句"你说的是哪个"
- IT 说"Claw 网关挂了" → 是 OpenClaw（路由层故障，所有 agent 受影响）
- CodeBuddy 设置里出现 Claw 选项 → 是 CodeBuddy 接 OpenClaw 的开关
- 看到 GitHub 上 "claw-code" → 是开源衍生项目（不等于 Anthropic 官方 CC）

为什么这么多 "Claw"

"Claw" 是腾讯内部围绕 AI agent 体系搭建的一套品牌前缀，含义偏向"敏捷的、能动手的助手"。但它指代的**不是单个产品**——而是整个生态的不同部件。

类比：**"水龙头"**。OpenClaw 是水管总阀（路由）· Claw Code 是工厂里能拆能改的工业水龙头（CLI）· WorkBuddy Claw 是装在你家厨房的成品（桌面 agent）· CodeBuddy Claw 标签是水龙头开关上的一个挡位（插件特性）。

记一句：OpenClaw ≠ Claw Code ≠ WorkBuddy Claw ≠ CodeBuddy 的 Claw 标签。下次听到先确认"哪一层"。

讲师备注 · 这是核心 framing 页。同事最容易混的就是这几个 "Claw"。可以让 1 位听众复述一遍 4 类区别 —— 复述对了说明吃透了。

Harness 工程 + Hermes 框架

两个"业内热词" · 听过别蒙 · 看完知道在哪一层

Harness 工程 · "怎么跑"

不是产品 · 是一套 framing

套在 AI agent 外面那层**管理框架**——决定 agent 如何被约束、如何运转、如何交付结果。

三件事的对照

工程	关注什么
Prompt 工程	怎么"说" · 写好 instruction
Context 工程	给"什么" · 管理 context 内容
Harness 工程	怎么"跑" · 编排 / 权限 / 多 agent / 反馈循环

典型例子：Claude Code 的源码（46k 行 QueryEngine + 40 个 permission-gated tools + 多层 hook）就是个 Harness。CodeBuddy 内部也有 Harness 层管多 agent 协作。

Hermes Agent · 开源具体产品

Nous Research · 2026.02 开源 · MIT 协议

"会学习、会沉淀的自进化 agent"—— 被誉为 OpenClaw 的开源替代方案。

核心特性

- 持久记忆：本机 `~/.hermes/`，关机不丢，跨会话保留
- 自我学习循环：从过去任务失败中学，下次自动调整
- 40+ 内置工具：文件 / 浏览器 / shell / API 调用
- Skill 自动生成：完成任务后自动写可复用 skill 文档
- 支持 MCP server 模式：和下一页要讲的 MCP 协议互通
- 多平台接入：飞书 / 钉钉 / Telegram / QQ 等 12+ 平台

对我们的意义：不一定要用，但要知道——业界正在把 "agent 自进化"做成标准能力。WorkBuddy 的 daily memory 就是这个方向的腾讯版。

记一句：Harness 是**视角**（怎么管 agent），Hermes 是**产品**（一个开源 agent 框架）。听到 "Harness" 想 "外壳"，听到 "Hermes" 想 "OpenClaw 的开源同行"。

讲师备注 · 这页是行业视野扩展。讲 30 秒带过即可。重点让听众对"Harness 工程"这个词有印象——以后看技术文章不蒙；对 Hermes 知道是 "OpenClaw 同类开源框架"即可。

MCP · 让 AI 连上"外部世界"的协议

Model Context Protocol · Anthropic 2024 年开放 · 现已成业界事实标准

问题：AI 默认是"封闭脑袋"

没有 MCP 之前，每个 agent 想接 Notion、GitHub、腾讯文档——都要 各自写一遍接入代码。
100 个 agent × 100 个服务 = 10000 份重复实现。

业务同事的痛：CodeBuddy 接了 Notion · CC 没接 · 你只能在两个工具间复制粘贴。

解法：MCP 是"AI 界的 USB-C"

MCP 定义了一套**统一协议**：服务方写一次 MCP Server，所有支持 MCP 的 agent 都能直接接。

- **MCP Server** = 服务方写的"插头"（如 notion-mcp-server）
- **MCP Client** = AI agent 内置的"插孔"（CodeBuddy / Claude Desktop / Cursor / Hermes 都有）
- 插上 → AI 能列出、读取、操作那个服务的数据



对业务同事的意义：装一个 Notion MCP，CodeBuddy 和 Claude Desktop 都能用，不用学两遍。**对 IT 的意义：**公司只要做一次 "腾讯文档 MCP Server"，全员所有 agent 受益。**第 2 章插件矩阵**会列出我们能直接用上的 MCP。

讲师备注 · 这页放在第 1 章压轴。OpenClaw / Claw 家族 / Harness · Hermes / MCP 形成"基础设施 → 视野 → 协议"的递进。下一章直接讲三同事用法时，所有概念已就位。

数据安全 · 隐私 · 合规边界

什么能喂 AI · 什么不能 · 出事怎么办 —— 销售第一个会被问的问题

✅ 可以喂 AI 的

- 公开行业资料 / 公开政策文件
- 客户案例（脱敏（去掉真实姓名/手机/合同金额，用 A 公司/B 客户代替）后的版本）
- 我自己的工作产出（提案模板 / 通话纪要骨架）
- 行业拓展指引 / 内部已发布的培训资料
- 开源代码 / 公开技术文档

❌ 不要喂 AI 的

- 客户真实姓名 / 手机 / 身份证 / 合同金额
- 内部薪酬 / 考核 / 组织架构未公开信息
- 未公开的产品策略 / 定价方案
- 其他客户的数据（用于和当前客户对比）
- 带水印 / 标注"内部"的文件原图

模型走向 · 三同事各走哪条路

数据敏感度	建议路径	三同事选谁	风险
高敏（客户真名/金额）	内网模型（走腾讯内部混元/自研模型，数据不出公司） · 必须脱敏后才上	CodeBuddy（内网可控）	高
中敏（脱敏案例/复盘）	OpenClaw 路由内部模型	WorkBuddy 走 OpenClaw 内部线 / CodeBuddy	中
低敏（公开资料 / 个人产出）	任意模型均可	CC / WB / CodeBuddy 都行	低

出事处理 SOP：① 立即停止使用当前会话 → ② 截图保留对话上下文 → ③ 报合规 / 法务团队 → ④ 不要私自删数据（删了反而难追溯）。数据驻留（数据存储和处理在哪个国家/地区，关系到管辖法律）问题不确定时，默认走内网模型最稳。

讲师备注 · 这页是销售视角必备。可问 "上周谁把客户合同金额发给过 AI？" 大概率有人举手。强调"脱敏不难，A 公司 / B 客户即可"。能答清楚客户更信任。

Claude Code · 速览

Anthropic 官方 CLI · 终端里的"动手干活"工程师



强项

- 读写本地文件、跑 shell 命令
- 跨多文件批改、git 操作
- 能"读图"（截图直接发它）
- 支持 **hook**（Hook：在写完文件、运行命令等时机自动触发后续脚本）：写完文件自动触发后续动作
- 支持 **skill**（Skill：把可复用工作流封装成模板，下次一句话调用）：把工作流封装成模板



局限

- 跨会话失忆（除非你写 `CLAUDE.md`）
- 没有日历 / 邮件 / 业务历史
- 不擅长跨日叙事（"我们三个月在做的事"）
- 计费按量 token，长上下文成本快速上升

什么时候叫它

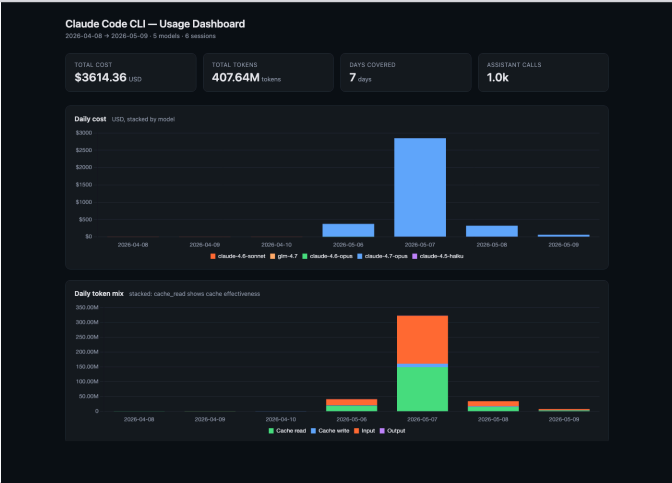
"我需要在我电脑上动手做一件具体的事" —— 改代码 / 整理一批文件 / 跑脚本 / debug / 出 HTML 单页。

一句话定位：CC 是"我的实习工程师"，给具体任务、给路径、给约束，它直接动手。

讲师备注 · 强调 "终端" 这个词业务同事可能怕。安抚 "你只要会输入中文就行，就像微信打字"。

CC Demo · 你的 token 花哪了

真实截图 · "本月用量 + 任务分布" 单页面板



CC 一句话生成 · 本机 ccusage 数据 · 单 HTML 文件可分享

怎么做的

- 跟 CC 说: "读 `~/claude/usage.json` , 生成单页面板"
- CC 自动: 读文件 → 算分布 → 写 HTML → 起本地 server
- 调一轮颜色和字号 → 截图 → 发群

耗时

从开口到出图, 约 15 分钟。换我手写要 1 小时。

关键点: 这种"读本机文件 → 算 → 出页面"的活, CC 是首选。它能直接动手, 不需要你来回粘贴数据。

WorkBuddy · 速览

业务场景定制人格 · 自动跨日记忆 · "长期顾问"



强项

- **自动持续记忆**：每次会话自动写 daily memory
- 默认上 **1M context**（模型一次能装在脑子里的信息量。1M $\approx$ 5 本书 / 半年对话），记得住半年事
- 有 **SOUL.md**（角色人格定义文件，决定 AI 的语气、立场、边界） · IDENTITY · USER 三件套人格
- 跨工作区共享记忆，跨日叙事最强



局限

- 慢（深思考型，5–30 秒/轮）
- 贵（默认 Opus + 1M）
- 不擅长在你电脑上"动手"（不主动跑命令）
- 需要持续投喂 1–2 周才显威力

什么时候叫它

需要"想得深、连得上、记得住"的活。复盘、季度规划、客户故事重组、跨多个项目找规律。

一句话定位：WorkBuddy 是"我的长期顾问"，要给它喂背景、喂偏好、喂历史，它给你跨周/跨月的洞察。

三件套人格 · SOUL · IDENTITY · USER

为什么 WorkBuddy 体感"懂你"——它每次会话自动注入这三份文件

SOUL.md 人格 · 怎么说话 · 做事原则 作用区间 每次会话注入 system prompt 顶端 典型字段 Core Truths · 操作风格 · 边界 · 语气样本 影响 语气 / 立场 / 决断风格 例子 "专业 · 简洁 · 不让用户做选择题"	IDENTITY.md 名片 · 我是谁 作用区间 SOUL 之后注入 · 给"自我"一个锚点 典型字段 名字 · 形态 · vibe · emoji 影响 自称 / 签名 / 视觉符号 例子 "我叫小助理, 是搭档"	USER.md 用户档案 · 你是谁 作用区间 注入 · 让 AI 知道对面是谁 典型字段 名字 · 称呼 · 城市 · 工作 · 偏好 · 硬规则 影响 专业术语 · 业务背景 · 关系网 例子 "叫 nic · 深圳 · 商务服务行业销售"
---	---	--

三件套的"分工"

- SOUL = 性格 · 改它 = 换人
- IDENTITY = 自我认知 · 改它 = 改名/改人设
- USER = 对方档案 · 改它 = 换服务对象

三个文件分开写而不是合一，是为了"局部修改不影响整体"——比如你换工作只改 USER.md，性格不变。

为什么 CC / CodeBuddy 没有?

- 定位不同: CC 是"工程师工具"·CodeBuddy 是"编辑器助手"·WorkBuddy 是"业务顾问"——只有顾问需要"长期人设"
- 架构不同: CC/CodeBuddy 主打多人协作（不绑定个人），WorkBuddy 是单用户独占实例
- 成本不同: 三件套每次会话都要注入 system prompt，吃 token，CC/CodeBuddy 默认 Sonnet 成本敏感
- 替代品: CC 可以用 `~/claude/CLAUDE.md` 模拟一份，但需要手写 + 手维护 · CodeBuddy 用 IDE 的"用户规则"达到 70% 效果

讲师备注 · 这页是 WorkBuddy 价值最深的解释。强调"局部修改"——为什么分三份。同事问"我也想让 CC 像 WB 一样懂我" → 答 "CLAUDE.md 写得越细越接近，但不会自动累积，要手动迭代"。

WorkBuddy Demo · 客户复盘 半小时出 v0

同一个客户 · 两个视角 · 同一份数据 · 不同呈现



对外版 · 给客户看 · 突出价值 · 留白多 · 数字克制



对内版 · 给 Leader 看 · 突出问题 · 数据密集 · 直接打分

关键差异：WorkBuddy 知道 "对外/对内" 不只是"换个标题"——是整套语气、详略、数据敏感度都要换。这种"懂场景"的能力靠的是它持续读你过去 10 个版本的复盘。

讲师备注 · 这页停 1 分钟让大家对比看。这是 WorkBuddy 价值最直观的 demo。

CodeBuddy · 速览 + vs WorkBuddy

同样是聊天式 AI · 但场景、记忆、节奏完全不同

✅ CodeBuddy 强项

- 编辑器内即时补全 / 解释 (IDE 原生)
- 选中代码 → 右键 → 改 / 注释 / 翻译
- 支持 MCP (刚讲过的那个: AI 接外部服务的统一协议), 可接 Notion / GitHub / 腾讯文档 / IMA
- 内网可控、合规审批走通
- 学习曲线最低 (5 分钟上手)

⚠️ CodeBuddy 局限

- 不直接跑 bash (但能生成命令让你跑)
- 跨日记忆较弱 (靠 User Rules + 项目文件)
- 主要场景在编辑器里, 跳出 IDE 体验弱
- 批量改文件不如 CC 直接

CodeBuddy vs WorkBuddy · 5 个对位差异

维度	CodeBuddy	WorkBuddy	业务影响
场景	编辑器内 · 写文档/写报告中途随手问	独立桌面 · 专门"坐下来"和它聊事	CB 适合 workflow 中插入 · WB 适合阶段性深聊
记忆深度	项目级 (AGENTS.md) + 全局偏好 (User Rules) · 不自动累积	跨项目跨日 · 自动 daily memory · 三件套人格	同一个客户的连续 10 次复盘 · WB 记得住, CB 要每次喂背景
默认模型	Sonnet (性价比)	Opus + 1M context (深思考)	CB 快但浅 · WB 慢但深
交互节奏	秒级响应 · 适合"问一句答一句"	秒到几十秒 · 适合"想清楚再问"	CB 不打断流 · WB 鼓励暂停
扩展机制	MCP 接外部服务 (Notion/腾讯文档)	Skill 沉淀工作流 (双周会/客户复盘)	CB 强在"接什么数据" · WB 强在"按什么流程做"

什么时候用谁的判定: "我在写代码/文档中途突然要问一下" → CodeBuddy; "我要专门花 30 分钟想一件大事" → WorkBuddy。两个不冲突, 可以同时开着。

讲师备注 · 这页核心是消除"它俩看起来差不多"的误解。强调"打不打断当前流"——这是体感上最直接的差别。

CodeBuddy · 三种典型用法

不止补全 · 也不只是写代码

① 补全 / 解释

最入门用法

- 写一半自动补
- 选中按 explain
- 翻译注释为中文
- 3 秒搞定一个 if

② Chat + MCP

进阶用法

- "帮我查腾讯文档里 X 文件"
- "在 GitHub 提个 issue"
- "读 Notion 里的产品文档"
- 外部数据直接进 chat

③ 单页文档

业务同事最实用

- 对话生成 HTML 单页
- 素材提炼成可视化报告
- "做一个法律行业拓展指引"
- 导出 PDF / 截图发群

实操路径：第一周只用 ① 补全 / 解释；第二周加 ② MCP 接腾讯文档 / 知识库；第三周开始用 ③ 出单页文档。三周见明显效率差异。



CHAPTER 03

该用谁

三个同事不是替代关系。每个任务都有最适合的那一个。

能力对比表 + 扩展生态

同一张表把"内置能力"和"外挂方式"全部说清

维度	CodeBuddy	Claude Code (CC)	WorkBuddy
载体	IDE 插件	命令行 CLI	独立 agent 实例
最强场景	编辑器内补全 / 解释	批量改文件 / git	跨日策略 / 业务记忆
记忆能力	项目文件 + User Rules	CLAUDE.md 手动维护	自动持续沉淀（最强）
能否跑 bash	不直接	可以（核心能力）	不擅长
公司合规	内网可控	视模型而定	视模型而定
速度	快（IDE 即时）	中	较慢（深思考）
成本	低	中	高
扩展机制	MCP 协议接外部服务	Skill + Hook	Skill + 长期记忆

外挂矩阵 · "它能做的事远超内置"

CodeBuddy · MCP 生态（连"外部服务"）

Notion

GitHub

腾讯文档

IMA 知识库

QQ 邮箱

TAPD

腾讯会议

+ N 持续扩

机制：协议层接外部数据 · 装一次全 agent 受益

WorkBuddy · Skill 体系（封" workflow"）

pdf

pptx

docx

xlsx

agent-browser

playwright

find-skills

NeoData

bs-ads-biweekly

客户复盘

机制：SKILL.md 文档化 workflow · 一句话调用

核心区分：MCP 解决"AI 能拿到什么数据"（外部连接），Skill 解决"AI 按什么流程做"（内部 workflow）。两者互补 —— 双周会用 Skill 套模板，但模板里要的数据通过 MCP 拉。

讲师备注 · 这页是 P16 + P18 融合版。把"对比表"和"插件矩阵"合到一页，避免重复。强调最后那句"互补"。

"我该用谁" · 决策矩阵

4 维需求 × 利弊分析 · 比单一决策树更全面

任务特征	推荐工具	为什么	替代方案 + 利弊
 批量改文件 / git 操作	CC	能直接读写本地 + 跑 shell · 跨多文件批改最快	CodeBuddy : 可生成命令但要你自己跑 · WB : 不擅长动手
 跨日策略 / 季度复盘	WorkBuddy	1M context + 自动记忆 · 记得住半年事	CC: 手写 CLAUDE.md 模拟 · 累 · CodeBuddy : 项目级记忆够浅
 IDE 内即时补全	CodeBuddy	原生 IDE 集成 · 秒级响应 · 不打断流	Cursor : 体验类似但非内网 · CC : 要切到终端, 太重
 业务文档单页报告	CC 或 CodeBuddy	CC 能直接出 HTML + 起 server · CB 能边写文档边改	WB : 可以但贵 · 用它做"内容深度", 让 CC/CB 出"形式"
 涉敏感客户数据	CodeBuddy	内网可控 · 合规审批走通 · 默认走内部模型	CC/WB : 视模型而定 · 走 OpenClaw 内部模型 OK · 走 Claude API 不行
 接 Notion / 腾讯文档	CodeBuddy	MCP 生态最丰富 · 装一次复用	Claude Desktop : 装 MCP 也行但脱离 IDE · CC : 可装但要 CLI 配置
 反复做的工作流 (双周会/复盘)	WorkBuddy	Skill 沉淀 · 一句话调用 · 自带历史口径	CC : 能写 skill 但不带业务记忆 · CB : 可调用 skill 但不沉淀
 成本敏感批量任务	CC + Haiku 模型	按量计费 · Haiku 极便宜 · 适合分类/格式化	CB : 默认 Sonnet 也便宜 · WB : 默认 Opus 太贵不要

兜底原则: 从轻到重升级。先 CodeBuddy → 不行升 CC → 还不行升 WorkBuddy。**反过来不要**: 用 WB 做"翻译一句文案" = 100 倍成本干 1× 的活。

讲师备注 · 这页代替原决策树。优势: 每个场景都列了"替代方案 + 利弊", 让听众理解"为什么这个不那个"。可以让大家挑 2-3 行自己最常碰到的场景对号入座。



CHAPTER 04

模型与上下文

Opus 是顾问 · Sonnet 是员工 · Haiku 是实习生。Context 是脑子大小。

模型家族 · 用同事类比记一辈子

Claude 主流三档 · 选型不需要懂技术参数

Opus（Anthropic 旗舰模型，最强推理力，最贵）	
高级顾问 · 最贵 · 最稳	
用在	策略 / 复盘 / 关键文档
速度	慢（5-30 秒/轮）
成本量级	1×（基准）
不要用	简单查询、补全

Sonnet（Anthropic 主力模型，性价比最高的中档）	
资深员工 · 性价比最高	
用在	日常 80% 任务
速度	中（2-10 秒/轮）
成本量级	1/5（比 Opus 便宜 5 倍）
注意	主力模型，默认用它

Haiku（Anthropic 轻量模型，速度极快价格极低）	
实习生 · 快 · 便宜	
用在	分类 / 摘要 / 格式化
速度	快（<2 秒/轮）
成本量级	1/50（极便宜）
不要用	需要思考的复杂任务

实操建议：日常用 Sonnet；卡住或要做关键决策时换 Opus；批量低难度任务用 Haiku。WorkBuddy 之所以贵是因为它默认 Opus + 1M context。

讲师备注 · "1× / 1/5 / 1/50" 是量级，不是精确数字。CC 和 CodeBuddy 默认 Sonnet，WorkBuddy 默认 Opus —— 这是为什么 WorkBuddy 体感更聪明也更贵。

Context Window（模型一次能装在脑子里的信息量。摊不下的就放抽屉里它看不到） + 价格直觉

"AI 同事一次能装在脑子里的信息量" · 决定能跨多深的事

类比：办公桌

Context window = AI 桌上能摊开的资料量。摊不下的就放抽屉里，它看不到。

- **200K context**（普通版）≈ 一本 300 页书
- **1M context**（WorkBuddy 用的）≈ 5 本书 / 一个项目半年的对话

权衡：1M 适合"长期顾问"角色；不适合"一次性任务"——杀鸡用牛刀。

价格直觉表

任务类型	建议模型	相对成本
分类 / 摘要	Haiku	1×（基准）
日常写代码	Sonnet	~10×
关键决策	Opus（200K）	~50×
长期顾问	Opus + 1M	~100×

典型踩坑：用 Opus 1M 跑"翻译一句文案"——100 倍成本干 1× 的活。反过来用 Haiku 跑季度复盘也亏，质量崩盘要重做。

讲师备注 · 不要给精确数字（合规问题 + plan 不同价格不同）。"量级"是关键词。同事问"我用 CodeBuddy 要不要担心成本"→答"公司给买的，你随便用，但别用 Opus 写补全代码这种事"。

非 Anthropic 模型怎么用

GLM / DeepSeek / 混元 / GPT 都能进 · 但路径不一样

通过 OpenClaw 网关接入

- 智谱 GLM 4.7 · 国内合规可用
- DeepSeek 3.2 · 价格便宜、推理强
- OpenAI GPT 系列 · 通用
- 混元 / 自研模型 · 公司内场景

实战经验：同一个 agent 可挂多个模型，按任务切换。中文密集任务有时 GLM 比 Claude 体感更顺。

⚠ 选模型的常见误区

- **不是越新越好：**GPT 5.4 发布不代表你的任务该换
- **不是越大越好：**72B 参数模型未必比 7B 适合分类
- **中英文表现不同：**Claude 中文略弱于国产模型
- **合规优先：**内部数据要走内网模型

讲师备注 · 本页可以快过。重点是"模型 ≠ 智能"这个心智 —— 选对场景比追新更重要。

CHAPTER 05

配置即记忆

写一份 Markdown，每次会话自动注入。一小时投入，半年复利。

AGENTS.md（项目级配置文件，OpenAI/Cursor/CodeBuddy 共享格式。一份文件三家共吃） · Why 为什么是它

不是 README · 不是 CLAUDE.md · 不是 CodeBuddy User Rules · 是跨家共用的"项目身份证"

❌ 没有 AGENTS.md 时

- 每个 agent 写一份配置：CC 写 CLAUDE.md · CodeBuddy 写 User Rules · Cursor 写 .cursorrules
- 规则漂移：改了 CLAUDE.md 忘了同步 .cursorrules → 同一个项目两个 agent 行为不一致
- 新人接手：不知道哪份是真规则
- README 不顶用：README 是给"人"看的（介绍项目），AI 看了不会按它执行

✅ AGENTS.md 解决

- 单一来源：一份文件，CC + CodeBuddy + Cursor + Codex 全部自动加载
- 开放标准：OpenAI 推动，Anthropic / Cursor 跟进，事实上的"agent 项目规约"
- 给 AI 看的：用命令式语言（"用 X 不用 Y"），不像 README 是叙述
- 规则集中：项目身份 / 数据 schema / 反模式 / 部署流程 / 跨 agent 注意事项

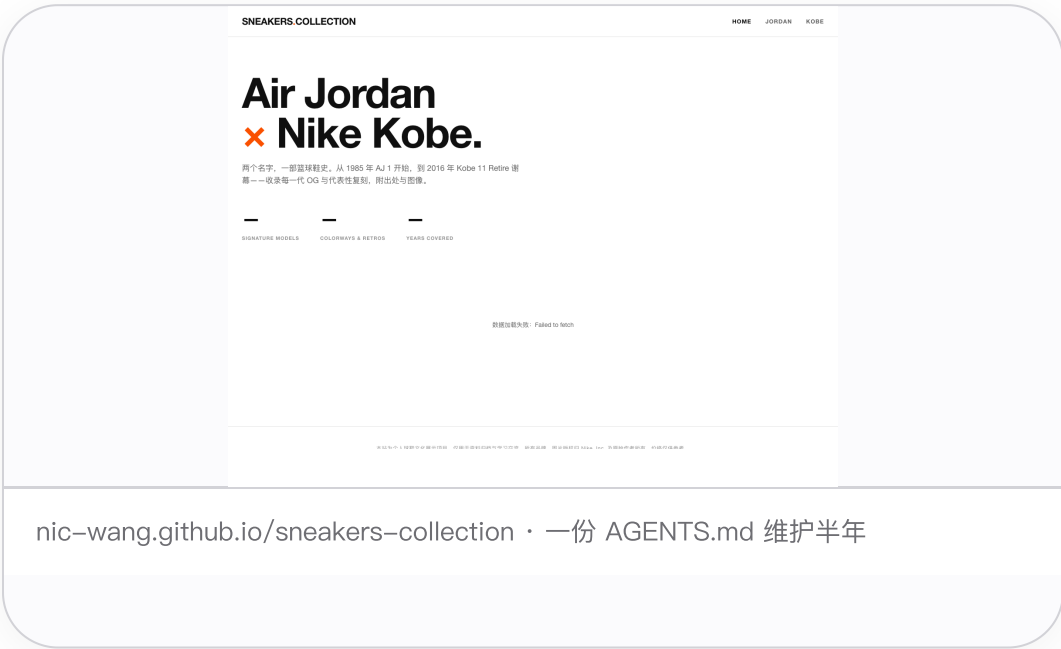
三层配置 · 各司其职 · 不要混着写

层级	位置	谁吃	写什么
全局偏好	~/.claude/CLAUDE.md + CodeBuddy User Rules	CC + CodeBuddy（手动同步）	身份 / 语气 / 输出格式偏好 / 反模式
项目规则	项目根/AGENTS.md	CC + CodeBuddy + Cursor + Codex 共吃	项目身份 · 文件地图 · schema · 部署 · 反例
业务记忆	~/.workbuddy/memory/	WorkBuddy 自动维护	对话沉淀 · 客户故事 · 决策历史（不要手写）

讲师备注 · 本页讲 why。强调"AGENTS.md 是给 AI 看的、是命令式的"。下一页讲 how。

AGENTS.md · How 怎么写

球鞋站项目实例 · 88 行 / 6 个章节 / 三家 agent 共吃



推荐章节骨架

1. **项目身份**：是什么、给谁用、技术栈、不变量 ("纯静态站、无构建")
2. **关键文件地图**：data/* · index.html · style.css · 各文件职责一句话
3. **数据 schema**：每个 JSON / 每张表的字段定义 (AI 改数据时按它走)
4. **反模式**：明确禁止 ("不要引入构建工具" / "不要改 schema" / "不要 inline style")
5. **部署流程**：怎么发布 ("git push → Pages 自动"), 失败怎么回滚
6. **常见任务模式**：典型工作流 ("加新鞋款 = 改 data/aj.json + 跑测试")

维护节奏

- 项目启动时写 v0 (30 分钟) · 跑通一次完整流程后回填遗漏的"不变量"
- 每次踩坑后追加"反模式"一行 · 半年下来形成项目知识库
- 不要让 AI 自己改 AGENTS.md (除非你确认) —— 它是"宪法", 要人审

✅ 好句式 (命令式 · 给 AI 看)

- "加新鞋款时 MUST 同时更新 data/aj.json 和 sitemap"
- "图片必须走 DuckDuckGo 抓取脚本, 禁止手动下载"
- "金额字段保留 ¥ 符号, 不要换成 RMB"

❌ 坏句式 (叙述式 · 不像规则)

- "这个项目主要做的是球鞋数据展示" → README 内容, AI 不需要
- "我们之前讨论过用 React, 但后来改了" → 历史叙事, 无用
- "如果可能的话最好不要..." → 模糊, AI 不知道是不是要遵守

讲师备注 · 这页讲 how. 强调"命令式 vs 叙述式"——这是同事最容易写错的点。带一份 88 行的实例, 给同事看完整结构。

Skill（Skill：把可复用 workflow 封装成 SKILL.md，下次一句话调用） · 把反复工作变模板

每周双周会汇总 / 客户复盘 / 报告排版 · 都可以一句话触发

已有 Skill 示例

- `bs-ads-biweekly-review` · 部门双周会汇总规约
- `bs-ads-quarterly-client-review` · 客户季度复盘（对外/对内双模板）
- `tencent-docs-precise-docx` · 精确排版上传腾讯文档
- `qq-email` · QQ 邮箱收发自动化
- `luckcraft-to-frontend` · 命理算法纯前端化

每个 Skill 一份 SKILL.md，描述触发词、流程步骤、踩坑备忘。

用法对比

✗ 没 Skill 时：每次双周会都要重新解释“用什么格式、强调什么、跳过什么”——5 分钟过去了。

✓ 有 Skill 后：一句“双周会”或“客户复盘”，AI 自动加载规约、套模板、按口径输出 v0。

（关键洞察：Skill 是“工作流的快捷方式”。如果某件事你一年要做 5 次以上，就值得 Skill 化。

讲师备注 · 强调 Skill 不是技术活——本质是把口头规约写成 markdown，AI 自动按它执行。任何一个销售都能写。

Hook（Hook：写完文件、运行命令等时机，自动触发后续脚本） · 让 AI 写完自 动接力

"AI 写完代码 → 自动跑测试" · "AI 写完 markdown → 自动同步到知识库"

三类常用 Hook

- **PostWriteFile**：写完文件后跑 lint / 测试 / 格式化
- **PreCommit**：提交前自动检查规范
- **OnSessionEnd**：会话结束自动归档 daily memory

Hook 用配置 JSON 写，CC 和 WorkBuddy 都支持。

为什么重要

Skill 解决"主动让 AI 做什么"，Hook 解决"AI 做完后自动做什么"。两者配合 → AI 真正接近"同事"——你给个起点，它自己跑完整条流水线。

我的实例：WorkBuddy 写完工作记忆后，hook 自动把它推送到 Obsidian 仓库 + 同步 iCloud。
我手动一次也不用动。

Skill / Hook · 最小可执行模板

复制即用 · 三步替换占位符 · 第二天就能用上

SKILL.md · 最小模板

```
---
name: customer-email-draft
description: 客户邮件初稿生成器
triggers: ["写邮件", "回复客户", "邮件初稿"]
---

## When to use
当需要给客户写正式邮件（提案/跟进/异议回复）
且希望保持我个人的语气和落款风格时。

## Steps
1. 询问对方角色（决策人/执行层）+ 邮件目的
2. 套用我固定模板：开头问候 → 1 句价值 → 3 点正文 → 行动召唤
3. 用克制中性语气，不夸张
4. 落款固定 "nic"，不加 emoji

## Output format
- Markdown · 主题行 + 正文
- 长度 200-400 字
- 关键数据用粗体标注
```

hooks.json · 写完自动同步

```
{
  "hooks": [
    {
      "name": "auto-sync-daily-report",
      "trigger": "PostWriteFile",
      "match": "reports/daily/*.md",
      "actions": [
        "prettier --write {file}",
        "tencent-docs upload {file} --space biweekly"
      ],
      "on_failure": "notify"
    }
  ]
}
```

写完 `reports/daily/*.md` 自动跑 prettier 格式化 + 推到腾讯文档"双周会"空间。再也不用手动同步。

三步替换占位符即可上手：① 把 `customer-email-draft` 改成你的场景名 · ② `triggers` 改成你会说的中文触发词 · ③ `Steps` 写成你的实际工作流。Hook 同理：改 `match` 匹配你的文件路径，改 `actions` 写要跑什么命令。

讲师备注 · 这页建议现场打开 IDE 改一行让大家看。强调"不是技术活，是把口头规约写成 markdown"。任何一个销售都能写 Skill；Hook 稍微进阶但也只是 JSON。

5 条误解辟谣 + 场景速查

现场可问可答 · 截图发群保存

5 条常见误解

- 1. "AI 会替代我"→不会，会替代不用 AI 的人
- 2. "我懂代码不能用"→正相反，业务同事最受益
- 3. "用 AI 会泄密"→看模型走哪个网关，合规模型 OK
- 4. "AI 不准"→大概率是没给上下文
- 5. "装一个就够"→三个 agent 对应不同场景

场景速查

- 客户邮件初稿→CodeBuddy
- 季度复盘→WorkBuddy
- 批量改文件名→CC
- 查腾讯文档→CodeBuddy + MCP
- 跨周策略思考→WorkBuddy
- HTML 单页报告→CC 或 CodeBuddy

第一性原则：先用，再调，再扩。不要一上来就追求完美配置。让"用了之后省了多少时间"驱动你下一步装什么。

讲师备注 · 这页给同事截图带走。下一页讲完整成长路径（Day 1 → Quarter 1）。

成长路径 · Day 1 → Week 1 → Month 1 → Quarter 1

四阶段曲线 · 不要跳级 · 上一阶段没跑通别上下一级



